Exercices de Compléments d'algorithmique

Série 4 – Techniques avancées d'algorithmique
(27 mars 2026)

Département d'Informatique – Faculté des Sciences – UMONS

Pré-requis : Notions d'orienté objet, générateur, techniques d'algorithmiques (cours jusqu'au **Chapitre 5**).

Objectifs : Appliquer diverses techniques algorithmiques sur un problème d'optimisation.

---

1 Travelling salesman problem (TSP)

Le « Travelling Salesman Problem » (ou « TSP ») est un problème d'optimisation bien connu en informatique. Il consiste à trouver, pour une liste de villes, le chemin le plus court passant par chacune des villes exactement une fois, et revenant au point de départ [1]. Le TSP a de nombreuses applications dans divers domaines (y compris l'informatique, la logistique, la construction de microprocesseur et bien d'autres).

De manière formelle, si nous considérons un ensemble V de n villes $1, 2, ..., n$ et la distance $d(i, j)$ entre la ville i et j, alors nous devons trouver la séquence $y_1, y_2, ..., y_n$ (où $\forall e \in V, \exists i \in \{1, 2, ..., n\}, e = y_i$ et $\forall i \in \{1, 2, ..., n\}, \forall j \in \{1, 2, ..., n\}, y_i \neq y_j$ si $i \neq j$) qui minimise

$$\left(\sum_{i=1}^{n-1} d(y_i, y_{i+1})\right) + d(y_n, y_1) \tag{1}$$

Remarquez que $y_1, y_2, ..., y_n$ représente une permutation des nombres de 1 à n (nos villes) (i.e., nous avons les mêmes éléments, mais nous avons changé leur position).

Intuitivement, vous devez trouver un tour (i.e., une séquence de villes partant de i et revenant à i) tel que la distance totale que vous effectuez est la plus petite possible. Comme le TSP suppose que vous revenez à votre point de départ, vous ne devez pas oublier de compter la distance entre la dernière ville que vous visitez et votre point de départ (la deuxième partie de la formule). Vous devez passer par toutes les villes (première condition) précisément une fois (seconde condition).

Par exemple, si nous avons 4 villes $V = \{1, 2, 3, 4\}$ avec la matrice de coût suivante :

	1	2	3	4
1	0	3	1	7
2	4	0	9	4
3	3	15	0	6
4	12	9	8	0

Cette matrice indique tous les coûts $d(i, j)$. Par exemple, pour avoir le coût pour aller de 1 à 3 (noté $1 \to 3$ par la suite), nous regardons la position $(1, 3)$, donc $d(1, 3) = 1$. Pour $3 \to 4$, nous regardons $d(3, 4) = 6$, etc. Via cette matrice, nous devons trouver un tour avec la distance minimale. Si nous choisissons que notre ville de départ est 1, alors nous devons trouver un tour partant de 1, passant par toutes les autres villes (2, 3 et 4) et revenant à 1. Ici, le plus petit tour est $y_1 = 1 \to y_2 = 2 \to y_3 = 4 \to y_4 = 3 \to y_1 = 1$. Son coût est $d(1, 2) + d(2, 4) + d(4, 3) + d(3, 1) = 3 + 4 + 8 + 3 = 18$ (qui est le plus petit que nous pouvons avoir ici). Notez que nous passons par chaque ville une seule fois et que

1. Le TSP est prouvé comme étant un problème NP-difficile. On ne connaît actuellement aucun algorithme en temps polynomial donnant toujours la meilleure solution pour ce problème. Il est aussi possible (mais pas encore sûr) qu'il n'existe aucun algorithme de ce genre.

nous revenons à 1 (notre ville de départ). Notez aussi que $1, 2, 4, 3$ (notre tour sans retourner au point de départ) est une permutation des villes de 1 à $n = 4$.

La compréhension du problème faisant partie du TP, vous êtes encouragé à vous documenter sur celui-ci si le problème n'est pas clair pour vous.

2 Un peu de mise en place

En voyant le problème du TSP, vous vous souvenez bizarrement du capitaine Edward. Vous vous dites que, si vous le souhaitez, vous pouvez devancer le capitaine Edward et récolter tous les trésors à sa place. Cependant, vous devez être efficace. Vous n'avez pas encore les trésors, vous ne voulez pas dépenser de l'argent inutilement, surtout que la fiabilité de la carte du capitaine est encore à vérifier. Vous vous dites que vous allez regarder quand même. Parmi les meilleurs trésors, vous allez chercher la meilleure route à prendre pour les récolter et revenir chez vous.

Ceci étant dit, il faut malgré tout faire un peu de mise en place avant de pouvoir aller récolter vos trésors en dépensant un minimum d'argent ! Simplement représenter le problème du TSP est déjà une bonne première étape vu que vous allez l'utiliser par la suite.

1 Commencez par créer une classe `TSP` qui représentera une instance du TSP. C'est à vous de définir les méthodes dont vous pourriez avoir besoin au cours du TP. Cependant, il vous est fortement conseillé de créer au moins des méthodes afin d'évaluer le coût d'une solution, d'avoir un coût entre deux villes données, ainsi que d'avoir le nombre de villes. Vous êtes libre d'en créer plus dépendant de votre vision des choses.

2 À des fins de test, créez une méthode générant une instance aléatoire du `TSP`. Les coûts sont aléatoires, et le nombre de villes peut être donné en paramètre de la méthode. Notez que le coût pour aller de $1 \to 2$ peut être très différent du coût pour aller de $2 \to 1$.

3 On brut force le tout

Vous voilà maintenant prêt à calculer vos tours de moindre coût. Vous vous dites premièrement que calculer le coût d'une solution est rapide. Tester toutes les solutions possibles devrait être jouable, pas vrai ? Ceci étant dit, il y a quand même beaucoup de combinaisons possibles. Mais ce n'est pas grave, vous allez tout de même essayer ! Vous voilà donc à essayer simplement de tester toutes les solutions possibles et de garder la meilleure (celle qui vous coûte le moins) [2].

3 Commencez par implémenter un générateur vous permettant, pour un ensemble de villes, d'avoir toutes les routes possibles.

4 Implémentez une approche par force brute pour résoudre le problème du TSP.
Conseil : Ne prenez pas des problèmes avec trop de villes si vous souhaitez que votre algorithme se termine avant la fin du TP.
Conseil : Vous allez implémenter plusieurs techniques pour le problème du TSP. Vous pouvez, si vous le souhaitez, créer une classe qui a pour but de résoudre le problème du TSP (`TSPSolver` par exemple).

5 Si vous considérez un temps maximum d'exécution (disons deux minutes), quelle est la taille maximum du problème que vous pouvez résoudre par force brute ? Autrement dit, quel est le nombre de villes maximum que vous pouvez considérer si vous utilisez la méthode force brute pour le TSP ?

6 À votre avis, quelle serait la complexité théorique dans le pire des cas de cette manière de faire ?

4 La méthode gloutonne

Vous vous êtes rendu compte que la force brute, c'est long pour les plus gros problèmes... Nous allons donc considérer une alternative : une méthode gloutonne.

2. Ce type d'algorithme est communément appelé des algorithmes de force brute ou « brut force » en anglais. (Basiquement, on teste toutes les possibilités pour trouver ce qu'on cherche). Ils sont souvent simples, mais très peu efficaces.

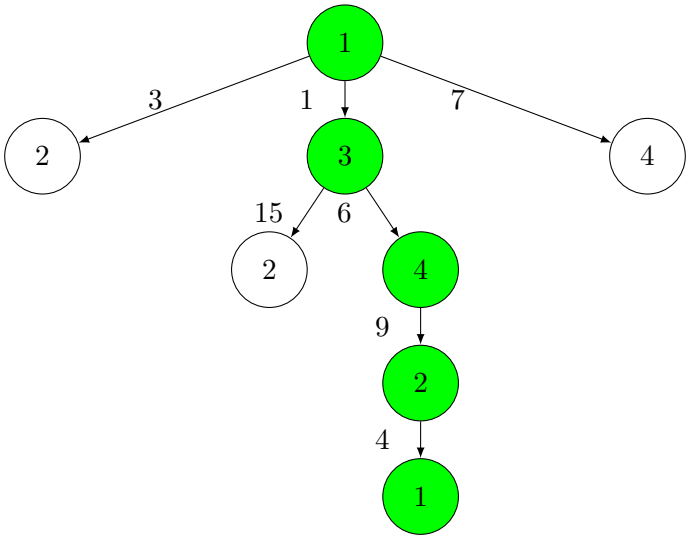


FIGURE 1 – Une illustration de l'algorithme glouton. Les flèches représentent les choix possibles à cette étape. Les nœuds verts sont ceux sélectionnés par la méthode gloutonne.

Pour rappel, la méthode gloutonne consiste à prendre la meilleure décision à chaque étape. Elles sont souvent rapides (car elles ignorent à chaque fois ce que d'autres décisions auraient donné), mais ne donnent aucune garantie sur la solution. La solution peut être optimale, comme elle peut être très mauvaise.

Dans ce but, considérons une solution gloutonne simple pour le TSP. Pour rappel, nous voulons générer une séquence de villes de manière à ce que faire le tour des villes dans cette séquence nous permette de parcourir une distance aussi petite que possible. En considérant que nous partons d'une ville y_0, nous pouvons choisir la ville y_1 en choisissant la ville la plus proche de y_0 (à l'exception de y_0 évidemment). Pour choisir y_2, nous choisissons à nouveau la ville la plus proche de y_1 qui n'a pas encore été sélectionnée (à l'exception de y_0 et y_1), etc. La Figure 1 illustre cet algorithme avec l'exemple donné en Section 1.

7 Implémentez la méthode gloutonne.

8 En résolvant plusieurs problèmes aléatoires, comparez le temps d'exécution et le coût du tour retourné par la méthode gloutonne et force brute. Voyez-vous une ou plusieurs différences significatives entre les deux méthodes ?

9 Essayez de créer une instance du TSP où la méthode gloutonne donne un résultat « aussi mauvais » que vous pouvez imaginer.

5 Programmation dynamique

Vous voilà ennuyé. L'un prend trop de temps de calcul, l'autre vous donne une solution pas toujours optimale. Essayons maintenant la troisième option.

La programmation dynamique consiste à résoudre des instances de vos problèmes en résolvant de plus petites instances de votre problème. Dans notre cas précis, nous voulons résoudre des problèmes du TSP avec n villes en résolvant des problèmes du TSP avec $m < n$ villes. Pour ce faire, nous allons voir le problème récursivement. Autrement dit, pour résoudre une instance du TSP à n villes, nous allons chercher à résoudre des sous-instances du TSP à $n - 1$ villes.

Imaginons avoir 5 villes (1, 2, 3, 4, 5). Si le tour $1 \to 2 \to 3 \to 4 \to 5 \to 1$ est le plus court tour passant par toutes les villes, alors $2 \to 3 \to 4 \to 5 \to 1$ est un des plus courts chemins allant de 2 à 1 passant par 3, 4, 5. Intuitivement, si ce n'était pas le cas, alors $1 \to 2 \to 3 \to 4 \to 5 \to 1$ aurait dû prendre le chemin le plus court afin d'être le plus court tour possible.

Résoudre une instance d'un problème du TSP avec l'ensemble V de n villes est donc simple à faire récursivement. Avant de lancer la récursivité, nous sélectionnons d'abord une ville v_0 dans V. (Le choix de cette ville n'a pas d'importance, ça peut être la première). Pour chaque ville v de V, nous

essayons de trouver le plus court chemin C passant par toutes les villes de $V - \{v\}$, débutant à v et se terminant à notre point de départ v_0. Une fois trouvé, nous ajoutons le coût du chemin $v \to C_0$ où C_0 est la première ville (la ville de départ) du chemin C. Si ce coût est le plus petit trouvé jusqu'ici, nous sauvegardons le chemin. Autrement, nous l'oublions. La récursivité nécessitant un cas de base, nous nous arrêtons quand $V = \emptyset$. Dans ce cas, nous retournons simplement notre ville de départ. (Le TSP suppose toujours qu'on retourne à notre ville de départ).

10 Implémentez une version récursive de la résolution du TSP. Vous pouvez vous aider des méthodes déjà faites jusqu'ici.

À l'instar d'autres problèmes récursifs, nous pouvons remarquer que nous résolvons parfois plusieurs fois le même problème. Par exemple, afin de calculer $1 \to 2 \to 3 \to 4 \to 5 \to 1$, nous avons aussi testé si $1 \to \mathbf{3} \to \mathbf{2} \to 4 \to 5 \to 1$ était le plus court chemin. Par notre récursivité, nous avons alors calculé le plus court chemin de 4 à 1 passant par 5 plusieurs fois. (pour toujours trouver $4 \to 5 \to 1$). Là où ici, le chemin ne concerne que trois villes (et est presque direct), vous pouvez imaginer qu'avec des problèmes plus grands, ce même principe s'appliquera aussi à des chemins plus grands. Simplement retenir les résultats déjà calculés par nos appels récursifs nous permettra de gagner du temps de calcul. Le fait de retenir les résultats intermédiaires est appelé la mémoïsation.

11 En appliquant le principe de mémoïsation, modifiez votre code afin de retenir vos résultats intermédiaires.

12 Voyez-vous une différence de temps d'exécution ?

Une seconde solution utilisant la programmation dynamique vous est donnée sur Moodle. Cette seconde solution se base sur le même principe de retenir des résultats intermédiaires. Elle implémente l'algorithme de Held–Karp [3], connu pour améliorer la complexité du TSP vers $\mathcal{O}(2^n n^2)$.

13 Comparez les différents algorithmes en temps d'exécution. Que remarquez-vous ?

14 Faites varier n pour chacun des différents algorithmes. Si vous prenez un temps maximum pour résoudre une instance (e.g., deux minutes), est-ce que les algorithmes utilisant la programmation dynamique vous permettent de résoudre des problèmes plus grands (voir nettement plus grand) ?

3. `https://en.wikipedia.org/wiki/Held\%E2\%80\%93Karp_algorithm`